AGL-SKILL: Coding Harnesses as Agentic Optimizers

Ryan Tsui* **Yuge Zhang** **Yuqing Yang**

Microsoft Research Asia

s2480214@ed.ac.uk Yuge.Zhang@microsoft.com Yuqing.Yang@microsoft.com

Abstract

Agent performance can often be enhanced by the use of skills: reusable workflows, code, and prompts that provide task-related information or guide the behaviour of the model. Current methods have improved upon handcrafting or one-shot generation by treating skill development as an optimization process, permuting the target skill iteratively in a way that concretely improves performance. In this paper, we explore the use of the models themselves as agentic optimizers: we give an agent a task and a train split, tell it to optimize the performance of a target model on said task, then evaluate it on a test split. Across five benchmarks, multiple models, three coding harnesses, and $5, $10, and $25 optimization budgets, we find that coding harnesses substantially improve target agents, while AGL-SKILL does not consistently outperform the same harness without a skill. The strongest optimizers often discover the same target-specific changes on their own. Our findings demonstrate the utility of coding harnesses as skill optimizers and characterize model behaviour as agentic optimizers.

1 Introduction

Coding harnesses allow Large Language Models (LLMs) to inspect files, use tools, and execute programs within a working environment. An agent can use the results of these actions to revise its approach and continue working on a task. For specialized tasks, however, it also needs information about the available tools, relevant procedures, and expected outputs.

This information can be provided through skills: reusable instructions, examples, and supporting code that guide an agent's behaviour. Skills can be written by hand or generated by a model, but deciding what to include requires knowledge of the task and the agent's limitations. Recent methods, including SkillOpt (Yang et al., 2026) and EvoSkill (Alzubi et al., 2026), treat skill development as an optimization process. They evaluate candidate skills, use task feedback to revise them, and retain changes that improve performance.

A coding harness already supports much of this process. It can inspect a target agent, run training examples, diagnose errors, and revise the instructions or code. What remains less clear is how well it performs as an optimizer without a dedicated skill, and whether additional guidance improves its results. Unlike an optimizer restricted to skill text, a coding agent can also change the surrounding implementation, including tools and execution logic.

*Work done during an internship at Microsoft Research Asia, Shanghai.

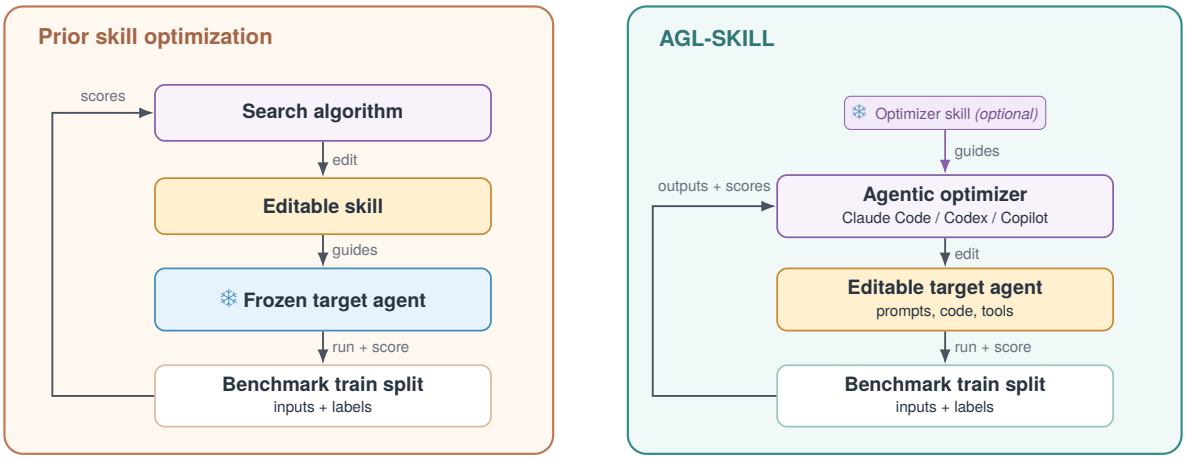


Figure 1: Comparison with prior skill optimization. Prior methods revise a skill while holding the target agent fixed. AGL-SKILL uses a coding harness to edit the target agent's prompts, code, tools, and runtime policy. We compare each harness with and without an optimizer skill under matched budgets. Snowflakes denote fixed components.

In this work, we introduce AGL-SKILL, a framework for evaluating coding harnesses as optimizers of other agents. Each optimizer receives an editable target agent, labeled training data, and a dollar budget. It can revise and test the target before the resulting agent is evaluated on a held-out split. We compare optimization with and without the Agent Lightning skill, matching the harness, target model, and nominal budget. Figure 1 summarizes this setup.

We evaluate Claude Code, Codex, and GitHub Copilot across five benchmarks covering spreadsheet editing, interactive tasks, question answering, and mathematical reasoning. Each condition uses a $5, $10, or $25 optimization budget. We report held-out accuracy alongside optimization and finale costs, and supplement these comparisons with separate experiments on subagent delegation and two-phase optimization.

Our results show that coding agents without an optimizer skill already substantially improve the unoptimized target agents. Adding the skill does not consistently improve accuracy. In the inspected runs, optimizers repeatedly improve input context, repair execution errors, and correct output formats. Many of these changes are found without an optimizer skill, suggesting that useful gains often come from basic improvements to the target implementation.

The main contributions are:

- An evaluation of coding harnesses as target-agent optimizers across five benchmarks, with no-skill controls, held-out accuracy, optimization cost, and finale cost.
- An analysis of how optimizer skills, subagent delegation, and two-phase optimization affect the changes made to target agents and their final performance.

2 Related work

Prompt and language-program optimization. Automatic prompt optimization treats instructions as parameters that can be improved from task feedback. APE generates and selects candidate instructions, OPRO conditions an optimizer on previous candidates and their scores, and ProTeGi combines natural-language critiques with beam search (Zhou et al., 2023; Yang et al., 2024; Pryzant et al., 2023). TextGrad generalizes textual feedback into a backward-pass abstraction, while GEPA evolves prompts through reflection and selection (Yuksekgonul et al., 2024; Agrawal et al., 2026). DSPy compiles declarative language-model pipelines against an

end metric, and MIPRO jointly optimizes instructions and demonstrations across multi-stage programs (Khattab et al., 2023; Opsahl-Ong et al., 2024).

Skill optimization. Skill optimization targets reusable agent guidance rather than a single task prompt. SkillOpt optimizes executive strategies (Yang et al., 2026), EvoSkill discovers skills and can generate executable helper scripts in multi-agent settings (Alzubi et al., 2026), and Trace2Skill distills trajectory-local lessons into transferable guidance (Ni et al., 2026). These methods are the closest published baselines for our evaluation. Our distinction is the editable surface: the coding harness may directly modify the target agent's prompts, tools, code, and runtime policy.

Agent and workflow optimization. Other systems expose a broader editable surface. AgentOptimizer treats an agent's functions as learnable parameters and updates them from execution histories (Zhang et al., 2024a). ADAS uses a meta-agent to program new code-defined agents, AFlow searches over code-represented workflow graphs using execution feedback, and Cognify tunes workflow structure, operators, and prompts under quality and cost objectives (Hu et al., 2024; Zhang et al., 2024b; He et al., 2025). These systems optimize more than prompt text, but each supplies a purpose-built search procedure over a particular agent representation.

Iterative and self-improving agents. Self-Refine revises outputs using self-generated feedback, while Reflexion carries verbal feedback across trials in episodic memory (Madaan et al., 2023; Shinn et al., 2023). STOP applies a language model to improve the scaffolding program that invokes it; SICA lets a coding agent edit its own implementation; and the Darwin Gödel Machine evolves an archive of self-modified coding agents (Zelikman et al., 2023; Robeyns et al., 2025; Zhang et al., 2025). Agent Lightning instead separates agent execution from reinforcement-learning training (Luo et al., 2025). AGL-SKILL evaluates an off-the-shelf coding harness as the optimizer of a separate target agent. Its primary comparison measures the additional value of a read-only optimizer skill relative to a no-skill harness under dollar budgets and held-out evaluation.

3 Method

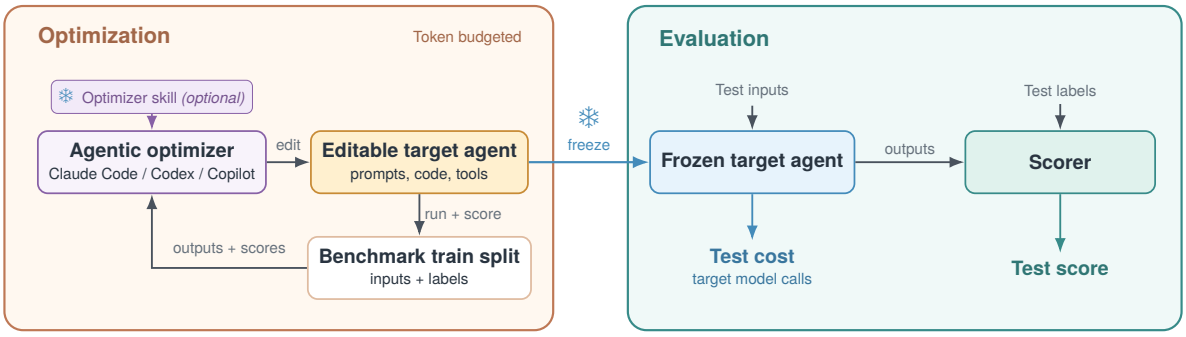


Figure 2: Optimization and held-out evaluation. The optimizer edits and scores a copy of the target agent using labeled training data under a dollar budget. The resulting target is frozen, run on label-free test inputs, and scored by a separate evaluator with access to hidden labels. Optimization and finale model costs are measured separately.

3.1 Implementation

Each coding harness optimizes a copy of the target agent. During optimization, it can inspect labeled training examples, run training evaluations, and edit the target's instructions, code, tools, and runtime policy within a dollar budget. The benchmark data and scorer remain fixed.

After optimization, we freeze the edited target and evaluate it on the held-out split (Figure 2). The target generates outputs from test inputs, and a separate evaluator scores them against hidden labels. Missing outputs count as incorrect.

Skill-based conditions mount a read-only optimizer skill. The published Agent Lightning skill is available in the `skills/agent-lightning/` directory of Microsoft Agent Lightning.

3.2 Experimental Setup

We evaluate Claude Code, Codex, and GitHub Copilot on five benchmarks. Each configured benchmark–harness–budget condition uses up to three repetitions. Vanilla and Accuracy → Cost are evaluated at nominal budgets of $5, $10, and $25. The Agent Lightning skill is reported on SpreadsheetBench, OfficeQA, and ALFWorld. Subagents and Git worktrees are evaluated at $10.

Codex and GitHub Copilot use `gpt-5.6-sol` with high reasoning effort. Claude Code uses `claude-opus-5`. The target model is `gpt-5.4-mini`, initially at medium reasoning effort. The optimization environment pins Claude Code CLI 2.1.175, OpenAI Codex SDK 0.144.1, and GitHub Copilot CLI 1.0.71.

The baseline is the original target agent before optimization. Vanilla gives the coding agent the optimization task without an optimizer skill. Every optimized target is frozen and evaluated on the held-out split.

We report mean held-out accuracy, optimization cost, and finale cost. Optimization cost includes optimizer calls and target-agent calls made during training and self-evaluation. Finale cost is the target agent's model cost over the full held-out test split.

3.3 Optimization variants

We evaluate five optimizer conditions:

Vanilla. The coding harness receives the optimization task, training data, and budget without an optimizer skill.

Accuracy → Cost. A two-phase skill maximizes training accuracy during the first half of the budget. During the second half, it reduces target-agent inference cost while preserving the best measured training score.

Agent Lightning skill. The coding harness loads the published Agent Lightning optimizer skill, which specifies the editable action space, evaluation loop, budget tracking, and checkpoint-selection criteria.

Subagents ($10). The optimizer is instructed to run two concurrent optimization threads from the same starting checkpoint in isolated workspaces. The parent is instructed to evaluate both candidates and select or revalidate a combined result.

Git worktrees ($10). The optimizer is instructed to create and evaluate candidate branches in isolated Git worktrees, then restore the best measured checkpoint.

3.4 Benchmarks

We evaluate on five benchmarks: SpreadsheetBench (Ma et al., 2024) for spreadsheet editing, ALFWorld (Shridhar et al., 2021) for interactive household tasks, OfficeQA (Databricks, 2025) for document-grounded question answering, SearchQA (Dunn et al., 2017) for question answering from retrieved snippets, and LiveMath (He et al., 2026) for mathematical reasoning. Table 1 lists

the splits and scoring methods.

Target	Task	Train/test	Score
SpreadsheetBench	Edit Excel workbooks by generating and executing Python	120/280	Official cell- and sheet-level comparison
ALFWorld	Complete text-based household tasks through sequential actions	3,553/134	Success on official `valid_unseen` games
OfficeQA	Retrieve Treasury Bulletin evidence and answer numerical questions	50/172	Normalized exact match
SearchQA	Answer questions from noisy retrieved web snippets	400/1,400	SQuAD-style exact match
LiveMath	Solve theorem-level five-choice mathematics questions	35/125	Exact choice accuracy

Table 1: Evaluation benchmarks. Training examples and labels are available during optimization; test labels remain hidden until final evaluation. Train/test gives the number of examples in each split.

4 Results

4.1 Overall performance

Table 2 compares the aggregate held-out accuracy of each optimization method. Vanilla coding agents improve on our baseline and outperform the strongest published method on every benchmark. Added optimizer guidance changes the outcome, but no guided condition improves consistently across targets. The optimized agents also differ in both optimization spend and deployment cost.

Every included Vanilla run improves on its corresponding baseline. The largest aggregate gains occur on SpreadsheetBench, ALFWorld, and LiveMath, where the original targets leave substantial room for changes to information flow, control logic, or reasoning policy. SearchQA begins from a stronger baseline and improves less.

Performance also depends on the pairing of harness and benchmark. No harness leads across every target or budget. ALFWorld approaches saturation for all three harnesses, while SpreadsheetBench and LiveMath retain larger differences between conditions. Detailed harness and budget results appear in Appendix A.

4.2 Effect of optimizer conditions

Added optimizer guidance does not produce a consistent accuracy gain. Accuracy $\rightarrow$ Cost is slightly stronger than Vanilla on OfficeQA and SearchQA in aggregate, but trails it on SpreadsheetBench, ALFWorld, and LiveMath. Across the traces, the accuracy phase generally finds the same benchmark-specific changes as Vanilla. The cost phase mainly lowers reasoning effort, narrows context, routes requests, or limits retries, and training-score preservation does not reliably transfer to held-out data.

The Agent Lightning skill follows a similar pattern. Its guidance does not change the dominant interventions: workbook grounding on SpreadsheetBench, stricter output and retrieval on OfficeQA, and deterministic control on ALFWorld. It slightly improves OfficeQA but trails Vanilla on SpreadsheetBench and ALFWorld, suggesting that general guidance affects search and validation more than the deployed change.

Method	Spreadsheet	OfficeQA	ALFWorld	SearchQA	LiveMath
Baseline	0.361	0.221	0.731	0.759	0.147
Human skill	0.429	0.459	0.567	0.772	0.288
LLM skill	0.368	0.366	0.657	0.761	0.280
Trace2Skill	0.407	0.209	0.828	0.786	0.328
TextGrad	0.382	0.300	0.709	0.775	0.272
GEPA	0.425	0.453	0.813	0.794	0.272
SkillOpt	0.475	0.488	0.858	0.802	0.328
Our baseline	0.277	0.322	0.578	0.764	0.261
Coding agents	**0.694**	0.539	0.998	0.824	**0.649**
Accuracy → Cost	0.636	**0.546**	0.944	0.825	0.595
with Agent Lightning skill	0.667	0.545	0.947	–	–
with subagents ($10)	0.565	0.525	0.968	0.819	0.643
with Git worktrees ($10)	0.662	0.513	**0.999**	0.818	**0.748**

Table 2: Mean held-out accuracy by method and benchmark. Published skill-optimization results appear above the divider; our unoptimized baseline and coding-harness conditions appear below. Values for our optimized methods average the available harness–budget condition means. Subagents and Git worktrees average the $10 Codex and Copilot conditions. Dashes indicate unavailable results.

Subagents broaden the search, but workers usually explore familiar alternatives, such as prompt and context changes versus deterministic tools or repair. The parent also spends budget establishing a shared baseline, comparing candidates, integrating changes, and revalidating the result. This coordination cost is consistent with weaker SpreadsheetBench and ALFWorld results; SearchQA and LiveMath remain close to Vanilla.

Git-worktree runs compare measured candidates and retain deterministic ALFWorld controllers. Because Vanilla already approaches the ALFWorld ceiling, they provide little evidence about broader gains from candidate isolation.

Overall, modern coding agents are already strong optimizers in this setting: without additional guidance, they repeatedly identify the same high-leverage changes. The tested skills and orchestration methods alter search and validation but do not consistently improve the final target.

4.3 Cost analysis

Optimization cost and finale cost measure different resources. Optimization cost covers the coding harness and its training evaluations, while finale cost measures the optimized target over the full held-out split. Table 3 reports both because spending more during optimization does not necessarily produce a more accurate or less expensive deployed agent.

Realized optimization spend often differs from the nominal budget. Some optimizers stop early, while a long model turn or evaluation can carry a run beyond its threshold. Across the Vanilla sweep, increasing the budget does not improve accuracy monotonically. Accuracy → Cost generally spends less at larger budgets, but these savings do not consistently preserve accuracy or reduce finale cost.

ALFWorld is the clearest case where optimization reduces deployment cost. Several optimizers replace model-driven action selection with deterministic controllers, producing near-perfect accuracy with zero or near-zero finale model cost. On the other benchmarks, lower finale cost is less consistent and must be considered together with held-out accuracy.

Lower realized optimization spend often reflects early stopping after a strong edit rather than deliberate cost reduction. Subagent and candidate-isolation conditions also devote part of their budget to coordinating and comparing candidates.

Method	Budget	Spreadsheet	OfficeQA	ALFWorld	SearchQA	LiveMath
Vanilla	$5	6.46 / 3.94	5.34 / 3.33	5.34 / 0.35	5.07 / 2.55	4.91 / 2.38
	$10	9.76 / 4.70	9.95 / 3.51	9.74 / 0.00	8.96 / 2.39	9.32 / 1.22
	$25	22.96 / 3.51	23.52 / 3.94	23.27 / 0.52	20.32 / 2.40	19.96 / 0.78
Accuracy → Cost	$5	4.49 / 3.76	4.78 / 3.21	4.24 / 0.92	5.07 / 2.30	3.87 / 1.22
	$10	9.97 / 4.98	7.35 / 3.44	6.33 / 0.69	5.94 / 2.47	4.75 / 1.29
	$25	16.04 / 5.16	17.26 / 3.33	12.25 / 0.73	10.24 / 2.51	8.00 / 1.08

Table 3: Mean realized cost in US dollars per run. Each cell reports optimization cost / finale cost, averaged over the available harness conditions at the stated nominal budget. Accuracy → Cost uses Codex and Copilot for SearchQA and all three harnesses for the other benchmarks.

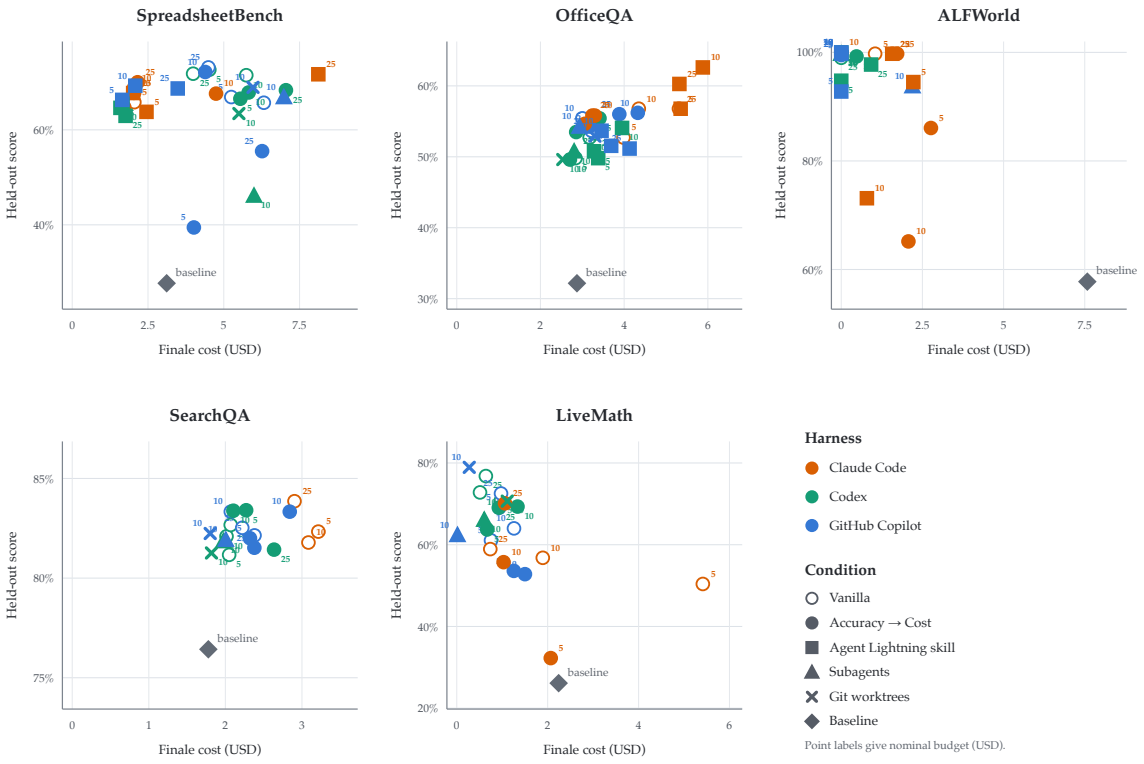


Figure 3: Held-out performance versus finale cost. Each colored point is the mean of up to three repetitions for one benchmark–harness–budget condition. Color denotes the harness, marker and fill denote the optimizer condition, and numeric labels give the nominal optimizer budget in US dollars. The gray diamond gives the mean score and evaluation cost of the pristine target. Finale cost covers the target over the full held-out split. Only available conditions are shown; axes are scaled separately by benchmark.

4.4 What the optimizers changed

The inspected runs show recurring edits within each benchmark. Many changes are confined to prompts and a few source files, although replacing model-driven control can require a larger implementation change.

SpreadsheetBench. Vanilla and skill variants expose workbook contents and structure to code generation, strengthen output instructions, and add repair after execution failures.

OfficeQA. Edits improve answer formatting, table and unit interpretation, and retrieval context. Some runs add document excerpts or calculator loops, but these additions do not consistently improve held-out accuracy.

SearchQA. Optimizers revise the prompt and normalize extracted answers, including short answer forms and strings found directly in the retrieved snippets. These changes recur across vanilla and two-phase runs.

ALFWorld. Optimizers replace model-driven action selection with deterministic controllers that parse the task and act through the environment's admissible actions. This changes the agent's execution logic and removes inference model cost in many runs.

LiveMath. Edits include more precise mathematical instructions and heuristics based on similarities between answer choices or the "stronger result" option. Some targets select an answer without a model call and use the model only for ambiguous cases.

5 Limitations and further work

Our benchmarks are unusually verifiable. Each provides labeled training examples, an automatic scorer, and a clear scalar objective. This supports rapid measurement and checkpoint selection, but may overstate performance in settings where feedback is delayed, noisy, subjective, or distributed across competing objectives. Clear benchmark structure can also reward narrow adaptations, such as deterministic ALFWorld controllers or LiveMath answer-choice heuristics, without demonstrating a broader improvement in agent capability.

A more general optimizer needs a systematic way to characterize the target before changing it. Useful measurements could include error clusters, scorer uncertainty, tool failures, cost and latency distributions, and the sensitivity of validation performance to individual changes. When exact labels are unavailable, trace-derived signals, constraint checks, pairwise preferences, or human feedback could provide intermediate objectives. These proxies would still require held-out validation because the optimizer may exploit an incomplete measurement rather than improve the intended behavior.

6 Conclusion

We evaluate coding harnesses as budgeted optimizers of target agents. Across five benchmarks, Vanilla coding agents improve every included run and, in aggregate, outperform the strongest published comparison on every benchmark. They repeatedly discover high-leverage, benchmark-specific changes, from workbook grounding and answer formatting to deterministic environment control.

Additional optimizer guidance and orchestration change how agents search and validate solutions, but do not consistently improve the final target. Larger budgets and lower optimization spend also do not reliably improve the deployed accuracy–cost tradeoff. Coding agents are already strong optimizers when progress can be measured directly; extending this approach to less verifiable settings will require faithful target-specific feedback.

References

Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. SkillOpt: Executive

strategy for self-evolving agent skills, 2026. URL https://arxiv.org/abs/2605.23904.

Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. EvoSkill: Automated skill discovery for multi-agent systems, 2026. URL https://arxiv.org/abs/2603.02766.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2211.01910.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2309.03409.

Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with Gradient Descent and beam search. In *Conference on Empirical Methods in Natural Language Processing*, 2023. URL https://arxiv.org/abs/2305.03495.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. TextGrad: Automatic "differentiation" via text. In *Advances in Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2406.07496.

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*, 2026. URL https://arxiv.org/abs/2507.19457.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines, 2023. URL https://arxiv.org/abs/2310.03714.

Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. In *Conference on Empirical Methods in Natural Language Processing*, pages 9340–9366, 2024. doi:10.18653/v1/2024.emnlp-main.525. URL https://aclanthology.org/2024.emnlp-main.525/.

Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Xiaoxi Jiang, and Guanjun Jiang. Trace2Skill: Distill trajectory-local lessons into transferable agent skills, 2026. URL https://arxiv.org/abs/2603.25158.

Shaokun Zhang, Jieyu Zhang, Jiale Liu, Linxin Song, Chi Wang, Ranjay Krishna, and Qingyun Wu. Offline training of language model agents with functions as learnable weights. In *International Conference on Machine Learning*, pages 60315–60335, 2024a. URL https://proceedings.mlr.press/v235/zhang24cd.html.

Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems, 2024. URL https://arxiv.org/abs/2408.08435.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating agentic workflow generation, 2024b. URL https://arxiv.org/abs/2410.10762.

Zijian He, Reyna Abhyankar, Vikranth Srivatsa, and Yiying Zhang. Cognify: Supercharging gen-AI workflows with hierarchical autotuning, 2025. URL https://arxiv.org/abs/2502.08056.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2303.17651.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2303.11366.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (STOP): Recursively self-improving code generation, 2023. URL https://arxiv.org/abs/2310.02304.

Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent, 2025. URL https://arxiv.org/abs/2504.15228.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents, 2025. URL https://arxiv.org/abs/2505.22954.

Xufang Luo, Yuge Zhang, Zhiyuan He, Zilong Wang, Siyun Zhao, Dongsheng Li, Luna K. Qiu, and Yuqing Yang. Agent lightning: Train any ai agents with reinforcement learning, 2025. URL https://arxiv.org/abs/2508.03680.

Zeyao Ma, Bohan Zhang, Jing Zhang, Jifan Yu, Xiaokang Zhang, Xiaohan Zhang, Sijia Luo, Xi Wang, and Jie Tang. Spreadsheetbench: Towards challenging real world spreadsheet manipulation, 2024. URL https://arxiv.org/abs/2406.14991.

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*, 2021. URL https://arxiv.org/abs/2010.03768.

Databricks. Introducing OfficeQA: A benchmark for end-to-end grounded reasoning. Official benchmark release, 2025. URL https://github.com/databricks/officeqa.

Matthew Dunn, Levent Sagun, Mike Higgins, V. Ugur Guney, Volkan Cirik, and Kyunghyun Cho. SearchQA: A new Q&A dataset augmented with context from a search engine, 2017. URL https://arxiv.org/abs/1704.05179.

Linyang He, Qiyao Yu, Hanze Dong, Baohao Liao, Xinxing Xu, Micah Goldblum, Jiang Bian, and Nima Mesgarani. LiveMathematicianBench: A live benchmark for mathematician-level reasoning with proof sketches, 2026. URL https://arxiv.org/abs/2604.01754.

A Detailed results

The following tables report mean held-out accuracy for each available benchmark–harness–budget condition. Each value averages up to three repetitions; dashes indicate unavailable results.

Vanilla

	Claude Code			Codex			Copilot		
Benchmark	$5	$10	$25	$5	$10	$25	$5	$10	$25
SpreadsheetBench	0.658	0.688	0.678	0.727	0.715	0.718	0.669	0.657	0.731
OfficeQA	0.527	0.568	0.568	0.498	0.527	0.533	0.541	0.554	0.537
ALFWorld	0.998	1.000	0.998	1.000	1.000	0.990	1.000	1.000	1.000
SearchQA	0.823	0.818	0.839	0.812	0.821	0.827	0.821	0.833	0.825
LiveMath	0.504	0.568	0.589	0.707	0.728	0.768	0.611	0.640	0.725

Accuracy → Cost

	Claude Code			Codex			Copilot		
Benchmark	$5	$10	$25	$5	$10	$25	$5	$10	$25
SpreadsheetBench	0.647	0.676	0.700	0.665	0.679	0.683	0.394	0.722	0.555
OfficeQA	0.547	0.558	0.558	0.554	0.496	0.535	0.543	0.562	0.560
ALFWorld	0.861	0.652	0.998	0.998	1.000	0.993	0.995	1.000	0.998
SearchQA	–	–	–	0.834	0.834	0.814	0.820	0.833	0.815
LiveMath	0.323	0.557	0.701	0.637	0.693	0.691	0.691	0.528	0.536

Agent Lightning skill

	Claude Code			Codex			Copilot		
Benchmark	$5	$10	$25	$5	$10	$25	$5	$10	$25
SpreadsheetBench	0.638	0.677	0.717	0.655	0.646	0.629	0.663	0.693	0.687
OfficeQA	0.568	0.626	0.603	0.498	0.541	0.508	0.516	0.537	0.512
ALFWorld	0.945	0.731	0.998	0.948	1.000	0.978	0.928	1.000	0.998

Subagents ($10)

Benchmark	Claude Code	Codex	Copilot
SpreadsheetBench	0.444	0.462	0.669
OfficeQA	0.494	0.508	0.543
ALFWorld	0.978	0.998	0.938
SearchQA	0.812	0.819	0.819
LiveMath	–	0.661	0.624

Git worktrees ($10)

Benchmark	Codex	Copilot
SpreadsheetBench	0.634	0.689
OfficeQA	0.496	0.529
ALFWorld	0.998	1.000
SearchQA	0.813	0.822
LiveMath	0.707	0.789

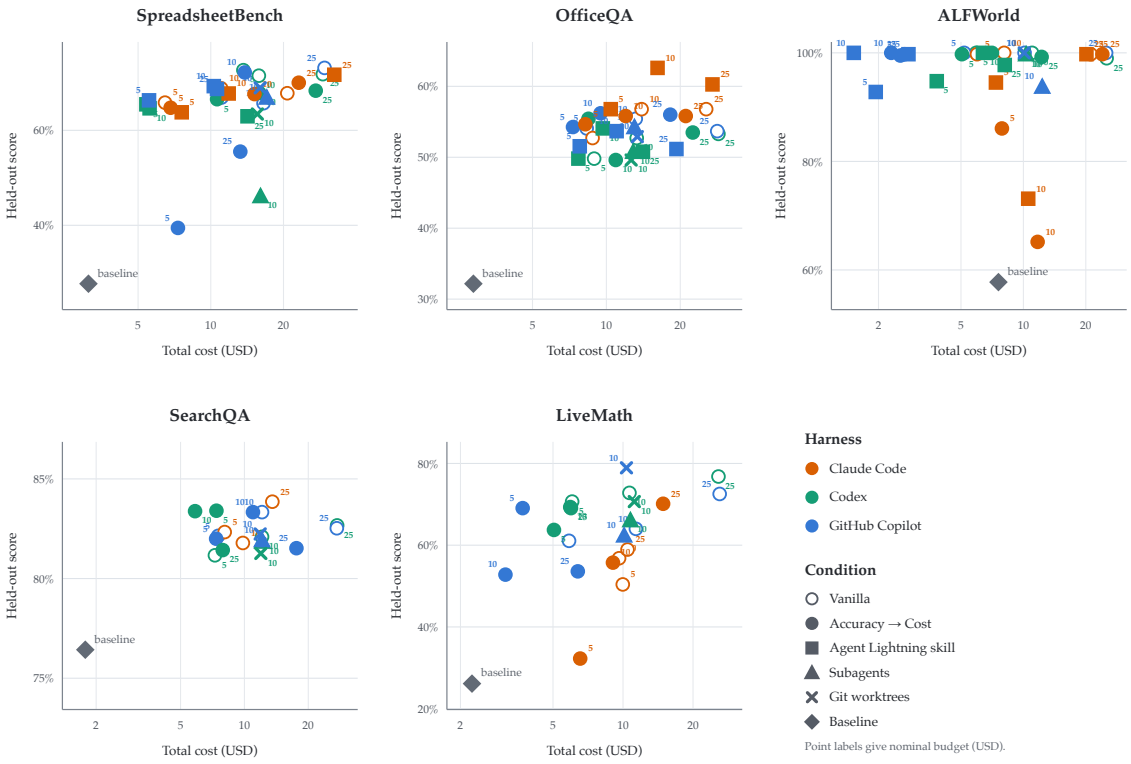


Figure 4: Held-out performance versus total run cost. For optimized conditions, total cost combines optimization and finale cost and excludes the separate pristine-baseline evaluation. The gray diamond reports that baseline evaluation itself. Other plotting conventions follow Figure 3; the cost axis is logarithmic.